



Capítulo 5

Objetivos:

- Explicar el concepto de separación por capas en el diseño de keywords y su impacto en la mantenibilidad de las suites de prueba.
- Describir las convenciones de nombres más utilizadas en proyectos Robot Framework profesionales y los principios que las sustentan.
- Interpretar la estructura de un proyecto bien organizado, identificando qué tipo de keyword pertenece a cada capa de abstracción.

**RF Avanzado — Buenas Prácticas,
Data-Driven y Extensión con Python**

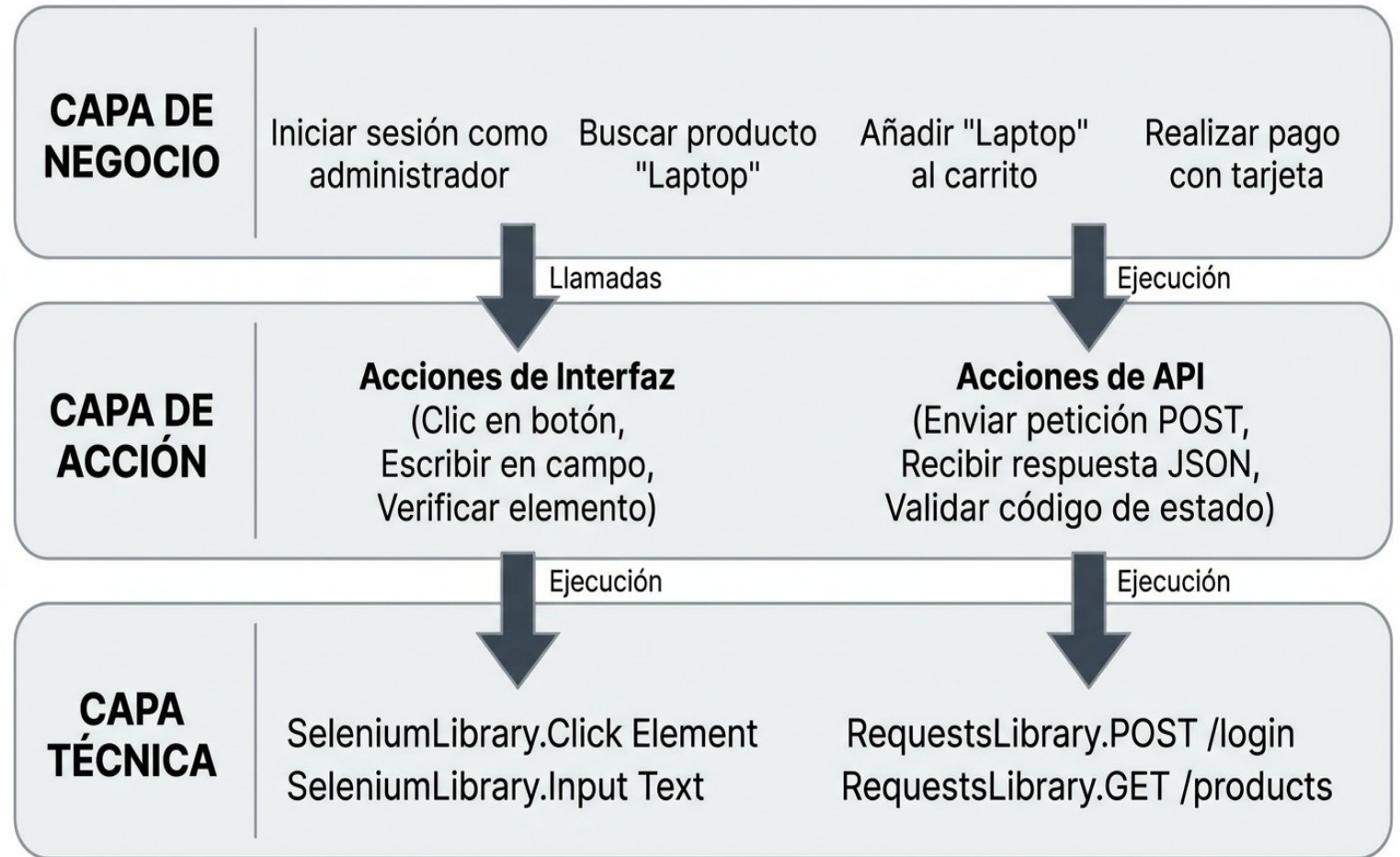
5.1: Diseño de Keywords Reutilizables: Separación por Capas y Convenciones de Nombres

- Introducción

El patrón de capas (técnica -> acción -> negocio) que viste en BDD aplica a cualquier suite. Convenciones de nombres consistentes reducen la curva de entrada de un equipo nuevo.

Arquitectura por Capas de Keywords

- ▶ Divide keywords en tres niveles de abstracción diferenciados.
- ▶ Cada capa tiene una única responsabilidad bien definida.
- ▶ Cada capa solo llama a la capa inmediatamente inferior.
- ▶ Reduce acoplamiento y facilita el mantenimiento.



Convenciones de Nombres para Keywords

Tabla comparativa de nombres de keywords

Nombre poco claro	Nombre recomendado
✗ login_proc	✓ Iniciar Sesión Con Credenciales
✗ chkUsr	✓ Verificar Estado Del Usuario
✗ do_action	✓ Ejecutar Acción De Compra

Menos legible

Claro y descriptivo

- ▶ Title Case: Verificar Que El Usuario Está Autenticado.
- ▶ Iniciar con verbo de acción: Obtener, Verificar, Completar, Navegar.
- ▶ Evitar abreviaciones crípticas — preferir nombres autoexplicativos.
- ▶ Elegir un idioma (español o inglés) y mantenerlo en todo el proyecto.

5.2: Data-Driven Testing: DataDriver con CSV/JSON y Trazabilidad con Tags

- Introducción

Cuando la misma lógica se valida con docenas de combinaciones de datos, DataDriver genera un test por cada fila de un CSV externo — separa la lógica de los datos.

Data-Driven Testing: Concepto y Motivación

- ▶ Separa la lógica de validación de los valores de entrada y salida esperados.
- ▶ Un solo caso de prueba se ejecuta N veces, una por cada fila de datos.
- ▶ Evita duplicar casos manualmente cuando hay docenas de combinaciones.
- ▶ Robot Framework nativo usa Test Template + filas en el mismo archivo .robot.
- ▶ El enfoque nativo mezcla datos y lógica, dificultando el mantenimiento.
- ▶ DataDriver externaliza los datos a CSV, JSON o Excel completamente.
- ▶ El plugin genera dinámicamente un caso independiente por cada registro.

Estructura Mínima de una Suite con DataDriver

```
# Instalación
# pip install robotframework-datadriver

*** Settings ***
Library      DataDriver      file=datos_login.csv      encoding=UTF-8

*** Test Cases ***
Verificar Login Con Datos Variables
    [Template]      Keyword De Login
    [Tags]          login      smoke

*** Keywords ***
Keyword De Login
    [Arguments]      ${usuario}      ${contrasena}      ${resultado_esperado}
    # Lógica de prueba (p. ej., llamada a API o UI)
    Log      Usuario: ${usuario} | Resultado esperado: ${resultado_esperado}
```

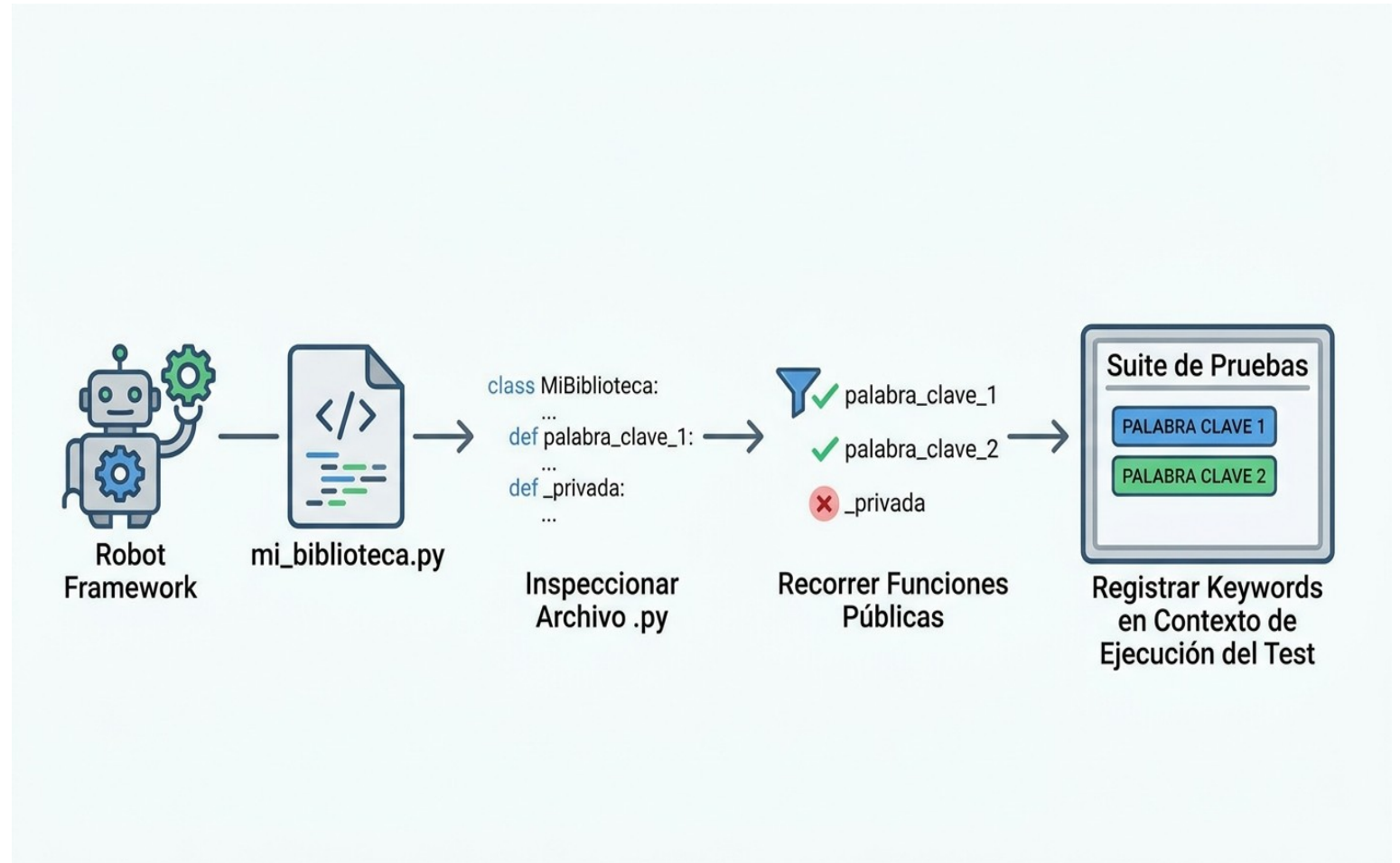
5.3: Librerías Personalizadas en Python: Keywords, Tipos, Documentación y Decoradores

- Introducción

Cuando las librerías nativas no alcanzan, escribes tus propias keywords en Python. Cualquier función pública se convierte automáticamente en keyword.

Keywords en una Librería Python

- ▶ Funciones públicas en un .py son keywords automáticamente.
- ▶ snake_case → Title Case sin configuración extra.
- ▶ RF localiza el archivo, instancia la clase y recorre métodos.
- ▶ Defaults, *args y **kwargs se mapean directamente.



Tipos de Librerías: Módulo, Clase y Estado

Tipo	Estructura Python	Estado entre keywords	Caso de uso típico
Módulo	Funciones en un.py	Sin estado	Utilidades puras, cálculos
Clase (TEST)	Clase con __init__	Por instancia / test	Conexiones, sesiones
Clase (SUITE)	ROBOT_LIBRARY_SCOPE='SUITE'	Persiste por suite	BD compartida en suite
Clase (GLOBAL)	ROBOT_LIBRARY_SCOPE='GLOBAL'	Única instancia global	Configuración global

Documentación de Keywords con Docstrings

- ▶ El docstring de la función es la doc oficial de la keyword.
- ▶ RF lo muestra en log.html, libdoc y tooltips del IDE.
- ▶ Incluir argumentos, retorno, excepciones y ejemplo de uso.
- ▶ Generar HTML: `python -m robot.libdoc MiLib.py MiLib.html`.

PÁGINA DE DOCUMENTACIÓN LIBDOC

NOMBRE DE LA KEYWORD → **HacerAlgo**

DESCRIPCIÓN → Esta keyword realiza la acción de...

TABLA DE ARGUMENTOS →

Nombre	Tipo	Valor por Defecto
arg1	cadena	""
arg2	entero	0

EJEMPLO DE USO → `HacerAlgo arg1="texto" arg2=10`

Decoradores: @keyword y Control Avanzado

PARÁMETRO	PROPÓSITO	EJEMPLO DE CÓDIGO EN LÍNEA
 name (str)	 Define el nombre clave de la funcionalidad para llamadas a la API.	<pre>@keyword(name="mi_funcion") def funcion(): pass</pre>
 tags (list[str])	 Asigna etiquetas para categorizar y filtrar la funcionalidad.	<pre>@keyword(tags=["core", "beta"]) def proceso(): pass</pre>
 tipos (dict[str, type])	 Especifica tipos de datos esperados para validación.	<pre>@keyword(tipos={"valor": int}) def calculo(valor): pass</pre>

- ▶ name=: cambia el nombre visible sin tocar la lógica.
- ▶ tags=: etiquetan keywords para filtrado y trazabilidad.
- ▶ Type hints activan conversión automática de str a int/float.
- ▶ Elimina llamadas manuales a int() o float() en la keyword.

5.4: Empaquetado, Versionado y Repositorios Compartidos de Librerías RF

- Introducción

Cuando un equipo crece, las librerías Python propias se benefician de empaquetarse (pip install) y versionarse con SemVer, en vez de copiarse manualmente entre proyectos.

Empaquetado de Librerías RF: Visión General

- ▶ Empaquetar una librería RF = convertir un módulo Python en un artefacto distribuable.
- ▶ El estándar PEP 517/518 define pyproject.toml como archivo central de configuración.
- ▶ pip install mi-libreria-rf descarga un .whl o .tar.gz y lo instala en el entorno activo.
- ▶ Robot Framework importa la librería con Library sin necesidad de rutas absolutas.
- ▶ El proceso requiere entender estructura de proyecto, pyproject.toml y herramientas build/twine.

Versionado Semántico (SemVer) en Librerías RF

- ▶ SemVer asigna significado a cada número: MAJOR.MINOR.PATCH.
- ▶ PATCH: corrección de bug sin cambiar la interfaz pública (ej. 1.2.0 → 1.2.1).
- ▶ MINOR: nueva keyword sin romper las existentes (ej. 1.2.1 → 1.3.0).
- ▶ MAJOR: keyword eliminada o firma modificada — rompe compatibilidad (ej. 1.3.0 → 2.0.0).
- ▶ Los equipos de QA fijan versiones en requirements.txt para controlar actualizaciones.
- ▶ SemVer comunica de inmediato si una actualización puede romper código existente.

Resumen del capítulo

- La **separación por capas** organiza las keywords en tres niveles: técnica, de acción y de negocio, reduciendo el acoplamiento y facilitando el mantenimiento.
- Cada capa debe llamar únicamente a la capa inmediatamente inferior, evitando dependencias cruzadas que dificulten los cambios futuros.
- Los **nombres de keywords** deben usar verbos de acción, escritura en Title Case y un lenguaje consistente (español o inglés) a lo largo de todo el proyecto.
- Una keyword bien nombrada es autoexplicativa: describe **qué hace**, no **cómo lo hace**.
- La estructura de carpetas del proyecto debe reflejar la separación por capas, ubicando cada tipo de keyword en su directorio correspondiente.



Resumen del capítulo

- Aplicar estos principios desde el inicio del proyecto evita la deuda técnica y reduce el tiempo de incorporación de nuevos miembros al equipo.
- El **data-driven testing** separa la lógica de prueba de los datos, permitiendo ejecutar un mismo caso con múltiples combinaciones sin duplicar código.
- El plugin **DataDriver** genera automáticamente un caso de prueba por cada fila del archivo de datos (CSV o JSON), manteniendo el archivo .robot con una única prueba plantilla.
- En **CSV**, los encabezados deben coincidir exactamente con los nombres de los argumentos de la keyword (incluyendo \${}); en **JSON**, se usan las mismas claves en un array de objetos.
- La columna especial [Tags] en el archivo de datos permite asignar tags por fila, habilitando filtrado, re-ejecución selectiva y **trazabilidad** directa hacia requisitos funcionales.



Práctica 9: Suite data-driven con CSV y segmentación por tags

Objetivo:

Implementar pruebas parametrizadas desde un CSV externo y generar reportes segmentados por tag de ambiente y prioridad.

Guía completa: [robfram-guide/Capitulo05/Practica9.md](#)



Práctica 10: Creación de librería Python personalizada e integración en suite

Objetivo:

Desarrollar un keyword personalizado en Python con documentación, probarlo con pytest e integrarlo en una suite.

Guía completa: [robfram-guide/Capitulo05/Practica10.md](#)



Referencias Bibliográficas

Capítulo 5: RF Avanzado — Buenas Prácticas, Data-Driven y Extensión con Python

- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>
- ▶ <https://robotframework.org/>
- ▶ <https://github.com/robotframework/robotframework>
- ▶ <https://robocorp.com/docs/courses/beginners-course-python>
- ▶ <https://github.com/Snooz82/robotframework-datadriver>